Research Statement - Marcelo d'Amorim

I develop **practical techniques that make software more reliable**, working at the intersection of **Software Engineering and Programming Languages**. Software bugs are costly and increasingly unavoidable, as ever more code is written by humans and synthesized by machine learning (ML). Through program analysis and systematic testing, my methods span the software quality lifecycle: bug prevention, detection, diagnosis, and repair.

Trajectory. I earned my PhD at the University of Illinois at Urbana-Champaign in 2007 and spent the following fifteen years building a research program in Brazil, where I was promoted to Associate Professor at the Federal University of Pernambuco in 2016. I joined NC State as a *tenured* Associate Professor in November 2022. This move was a deliberate step change rather than a continuation of the same trajectory: I rebuilt my group from the ground up (now six PhD students, with former students already holding faculty positions in the United States and Brazil), shifted the center of gravity of my funding to U.S. federal (NSF) and domestic industry sources, and reoriented my agenda toward AI. My group has moved from making established analyses practical to *defining new testing methodologies* for AI infrastructure and AI-enabled systems (Axis 2 below), domains that resist the precise specifications and oracles conventional testing relies on. This is where I place my greatest emphasis. Accordingly, this statement foregrounds work from the post-2022 period, situating it atop the foundation I built earlier.

Funding. My research has been supported by diversified funding totaling $1.78M over my career, including three grants from the U.S. National Science Foundation (NSF), one of them (my largest) held as sole PI and currently active, an industry grant from L3Harris, and three grants awarded through international competition. Since joining NC State in November 2022, my personal share of awards totals $956K.

Impact and Service. My group releases its methods as open-source tools we maintain,[1] and they have demonstrable impact: they have revealed hundreds of bugs in widely-used systems, including C/C++ compilers (e.g., GCC and Clang) [C3] and deep learning libraries (e.g., PyTorch and TensorFlow) [C1,C2], as well as security vulnerabilities in real-world code [W4]. My publication record comprises 52 full conference papers and 22 journal articles, including many at the field's most selective venues (e.g., ICSE, ASE, FSE, ISSTA, PLDI, and IJCAI). This work has been recognized with a best-paper award at ICST 2026, a Microsoft SEIF award, and Distinguished Reviewer awards at FSE and ASE. I also serve the field in leadership roles: on the program committees of its most prestigious conferences, on the ESEC/FSE Steering Committee, as a guest editor for STVR, and as General Chair of FSE 2024,[2].

VISION

My vision is that research earns lasting impact by being *practical*: it pursues fundamental questions in the service of real-world use, what Stokes termed *use-inspired basic research*, or "Pasteur's quadrant."[3] Many powerful analyses, such as symbolic execution, runtime verification, and model checking, are precise in principle yet too costly, too noisy, or too hard to specify to be adopted in practice. I design lightweight techniques that retain the essential guarantees of these analyses while fitting everyday development workflows, scaling to real-world software, and producing results developers can act on. I hold my work to the standard of adoption: my constraint solver `CORAL` [C40] was integrated into NASA's Symbolic PathFinder, and other researchers have independently built on my group's work in configurable-software testing [C39] and test flakiness [C22].

Rigorous experimental evaluation underpins my work. I treat a technique as only as credible as the evidence behind it, and I therefore assess my methods through controlled experiments on real-world software, following established empirical research methods[4] so that results are measurable, reproducible, and generalizable. In keeping with open-science standards, my group releases every project as open source, so others can independently reproduce and build on our findings.

The stakes are rising: as AI systems increasingly both write software and are themselves software, quality assurance must scale to code and models that no human fully understands. I intend for my group to help lead this shift, extending my agenda for practical, empirically grounded quality assurance to AI-generated code and AI-based systems.

ONGOING AND FUTURE WORK

My ongoing and future work advances two axes: **practical software engineering** and **testing of emerging, AI-centric domains**. The first axis sustains an agenda I have built over more than a decade; the second is a research direction I am now establishing, where I see the largest opportunity and place my greatest emphasis.

Axis 1: Practical Software Engineering. I develop lightweight techniques that improve everyday software reliability. In *runtime verification* (RV), which monitors software against formal specifications, my group has worked to lower its two main barriers to adoption: the flood of reported violations, which we triage with ML classifiers [C19,J10], and the difficulty of writing specifications, which we ease by supporting imperative styles and mining specifications from code [C18,C14]. Beyond these, we make monitoring faster and evolution-aware, re-checking only the properties a change affects, and we apply RV to detect misuses of security (crypto) APIs [C8,C5,J8]. In *regression testing*, I attack *flaky tests*, which non-deterministically pass or fail and erode trust in test suites: my group eliminates flakiness from parallelism [C28,C15], detects it both statically and by injecting noise [C22, C21,W3], and characterizes it across languages and computational environments [J11,J7]. I also build *developer-productivity tools* and study the defects that undermine reliability in practice: repairing broken Dockerfiles, searching for reusable API examples, and linting Infrastructure-as-Code for security [J16,C17,J9,C13], alongside empirical studies of configuration, supply-chain, and

[1] `https://github.com/ncsu-swat`

[2] Service: `https://damorim.github.io/service.html`

[3] D. E. Stokes. *Pasteur's Quadrant: Basic Science and Technological Innovation.* Brookings Institution Press, 1997.

[4] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén. *Experimentation in Software Engineering.* Springer, 2012.

portability defects in real projects [J1,W1,C4]. Increasingly, I use AI, including large language models (LLMs), to lower the cost of writing and maintaining tests: generating oracles [J6,J2], synthesizing fuzzing harnesses [C1,C2], hunting down test smells, and executing partially specified code [J4,C10]. Across these threads I plan to blend static and dynamic analyses and to keep humans in the loop and tools easy to adopt.

Axis 2: Testing Emerging, AI-Centric Domains. AI has become critical infrastructure, yet it is ultimately (complex) software and, like all software, is riddled with bugs. Worse, the software behind it is exceptionally hard to test: it rarely admits precise specifications or reliable oracles. This is where I place my greatest emphasis, along three fronts.

★ *Testing AI infrastructure.* Deep learning (DL) libraries such as PyTorch, JAX, and TensorFlow underpin modern AI, yet they are complex and buggy. Because their APIs lack precise input specifications, my group learns those specifications, symbolically and with ML, to synthesize valid, bug-triggering inputs [C1,C6]; complements this with coverage-guided fuzzing of library backends [C2]; and curated a database of reproducible DL-library bugs to ground future research [C9]. Together, these efforts are establishing systematic testing of AI infrastructure as a research direction; the resulting tools have already revealed hundreds of bugs, and I plan to extend them across frameworks and down to the compilers and kernels beneath them.

★ *Testing AI-enabled systems.* Autonomous systems, such as self-driving vehicles, must act safely in an open world where exhaustive testing is infeasible. My group exposes misbehavior efficiently by using learned signals to steer testing toward likely failures: ThirdEye reads a model's attention maps to flag unsafe decisions [C12], XMutant turns explainable-AI signals into targeted input mutations [J3], and Foresee predicts where a system will misbehave and focuses test generation there [C7]. Today these signals come from the system under test; looking further ahead, I want to lift the same principle to the system's *environment* through *world models*, learned predictive models of environment dynamics. A world model would let us simulate the rare, safety-critical scenarios that matter most and anticipate failures before deployment, turning the testing of AI-enabled cyber-physical systems from a reactive search into a principled, prediction-driven methodology.

★ *Measuring the self-confidence of AI.* A recurring obstacle across the above is that AI systems do not reliably know when they are wrong: AI-generated code, for instance, is often insecure or buggy [W2,J5], and even LLM-based repair remains unreliable [C11]. I am beginning to study how to measure and calibrate the *self-confidence* of AI models and AI-generated code, so that low-confidence outputs can be flagged for human review and testing effort steered toward them. I view this as a foundation for the trustworthy deployment of AI and the natural next step for my agenda on practical, empirically grounded quality assurance.

PAST WORK

This section summarizes the work I did, grouped by area. **Symbolic Execution and Constraint Solving [C45,J20,C42, C40,C37,C33]**. Symbolic execution generates test inputs by deriving constraints for distinct program paths and solving them, but constraints from general programs often build on undecidable or intractable theories that off-the-shelf solvers cannot handle, limiting the paths a tool can explore. We developed the constraint solver `CORAL` [C45,J20,C42,C40], which combines interval constraint solving with search to tackle complex constraints (e.g., involving floating-point variables and math functions), and `qCORAL` [C37,C33], a variant that quantifies solution spaces via Monte Carlo simulation to estimate the probability of program events. Both solvers are publicly available. **Program Model Checking [C49,W6,C47,W5,J21,J19].** State-space exploration is the essence of model checking, where time efficiency is central. Delta Execution (DE) [W6,C47,W5,J21] explores the search space with *sets* of concrete states, exploiting data similarity across states to speed up execution, backtracking, and state comparison; it improved exploration time up to 130x in the popular Java PathFinder model checker. Mixed Execution [C49, J19] uses a more efficient state representation for deterministic blocks, improving overall exploration time up to 37%. **GUI Testing [C46,C26].** In collaboration with Motorola, my team developed fuzzing techniques to find crashes and freezes in cell phones by exercising the UI of built-in apps [C46], comparing strategies based on recombining existing UI-test fragments and on exploring GUI components exhaustively or randomly. More recently, with a team from Georgia Tech, I helped synthesize Android Espresso (UI) tests from natural language bug reports to ease bug reproduction [C26]. **Runtime Verification (RV) [C52, W8,W7,C50,C19,C18].** RV monitors software properties at runtime and, beyond self-adaptation of safety-critical systems, has become practical during testing, revealing hundreds of bugs in code that violates Java API specifications. I explored efficient monitors [W8,W7,C50], ML classifiers to prioritize the many (often false-positive) violations [C19], and showed that developers tend to prefer imperative over declarative specifications [C18]. **Bug Finding in Configurable Software [C39,C34,C29,J17, J15].** Bugs can hide in specific configurations because the configuration space is often too large to test exhaustively. SPLat [C39] treats configuration choices as non-deterministic and uses symbolic execution to generate test inputs covering distinct configurations, finding bugs in complex systems such as the GCC compiler infrastructure [C34,C29,J17,J15]. **Security Testing [C44, C43,J18,W4,J12].** I proposed static information-flow analyses to identify vulnerabilities in PHP web applications (e.g., SQL injections) [C44,C43,J18], a method to flag vulnerable IoT devices by analyzing their controlling mobile apps [W4], and an algorithm to synthesize Network Intrusion Detector rules from malicious and benign traffic examples [J12]. **Software Fault Localization [C41,C38,C31,C30,C27,C24].** Fault localization identifies the program statements likely responsible for a failure. I explored techniques to improve their precision and developers' debugging effectiveness, combining statistical fault localization with change-impact analysis [C41] and incorporating human feedback [C31,C27]. **Test/Code Repair[C25,C17].** I explored repairing test cases [C25] and Infrastructure-as-Code scripts such as Dockerfiles [C17]. In both, the culprit is typically a change (e.g., an API update that breaks referring tests, or a "latest" version tag that forces a dependency upgrade) that automated refactorings cannot always absorb.

PUBLICATIONS

Journal Articles

[J1] Yue Zhang, Uchswas Paul, **M. d'Amorim**, and Akond Rahman. Configuration Defects in Kubernetes. In *IEEE Transactions on Software Engineering (TSE)*. 2026.

[J2] Facundo Molina, Alessandra Gorla, and **M. d'Amorim**. Test Oracle Automation in the Era of LLMs. In *ACM Transactions on Software Engineering and Methodology (TOSEM)*. 2025.

[J3] Xingcheng Chen, Matteo Biagiola, Vincenzo Riccio, **M. d'Amorim**, and Andrea Stocco. XMutant: XAI-based Fuzzing for Deep Learning Systems. In *Empirical Software Engineering (EMSE)*. 2025.

[J4] R. Melo, Pedro Simões, Rohit Gheyi, **M. d'Amorim**, Márcio Ribeiro, Gustavo Soares, Eduardo Almeida, and Elvys Soares. Agentic LMs: Hunting Down Test Smells. In *IEEE Software*. 2025.

[J5] Liliana Pasquale, Antonino Sabetta, **M. d'Amorim**, Péter Hegedűs, Mehdi Tarrit Mirakhorli, Hamed Okhravi, Mathias Payer, Awais Rashid, Joanna C. S. Santos, Jonathan M. Spring, Lin Tan, and Katja Tuma. Challenges to Using Large Language Models in Code Generation and Repair. In *IEEE Security & Privacy*. 2025.

[J6] Ishrak Hayet, Andrew Scott, and **M. d'Amorim**. ChatAssert: LLM-based Test Oracle Generation with External Tools Assistance. In *IEEE Transactions on Software Engineering (TSE)*. 2024.

[J7] Denini Silva, Martin Gruber, Satyajit Gokhale, Ellen Arteca, Alexi Turcotte, **M. d'Amorim**, Wing Lam, Stefan Winter, and Jonathan Bell. The Effects of Computational Resources on Flaky Tests. In *IEEE Transactions on Software Engineering (TSE)*. 2024.

[J8] Adriano Torres, Pedro Costa, Luis Amaral, Jonata Pastro, Rodrigo Bonifácio, Owolabi Legunsen, **M. d'Amorim**, Eric Bodden, and Edna Dias Canedo. Runtime Verification of Crypto APIs: An Empirical Study. In *IEEE Transactions on Software Engineering (TSE)*. 2023.

[J9] Brittany Reid, Keila Barbosa, **M. d'Amorim**, Markus Wagner, and Christoph Treude. NCQ: code reuse support for Node.js developers. In *IEEE Transactions on Software Engineering (TSE)*. 2023.

[J10] Lucas Cabral, Breno Miranda, Igor Lima, and **M. d'Amorim**. RVPrio: a Tool for Prioritizing Runtime Verification Violations. In *Software Testing, Verification and Reliability (STVR)*. 2022.

[J11] Keila Barbosa, Ronivaldo Ferreira, Gustavo Pinto, **M. d'Amorim**, and Breno Miranda. Test Flakiness Across Programming Languages. In *IEEE Transactions on Software Engineering (TSE)*. 2022.

[J12] Lucas Alcantara, Guilherme Padilha, Rui Abreu, **M. d'Amorim**. Syrius: Synthesis of Rules for Intrusion Detectors. In *IEEE Transactions on Reliability (TRel)*. March 2021.

[J13] Igor Lima, Jefferson Silva, Breno Miranda, Gustavo Pinto, and **M. d'Amorim**. Exposing Bugs in JavaScript Engines through Test Transplantation and Differential Testing. In *Journal of Systems and Software (JSS)*. November 2020.

[J14] Rohit Gheyi, Márcio Ribeiro, Beatriz Sousa, Marcio Guimarães, Leo Fernandes, **M. d'Amorim**, Vander Alves, Leopoldo Teixeira, and Baldoino Fonseca. Identifying Method-Level Mutation Subsumption Relations using Z3. In *Information and Software Technology (IST)*. November 2020.

[J15] Igor Lima, Jeanderson Candido, and **M. d'Amorim**. Practical Detection of CMS Plugin Conflicts in Large Plugin Sets. In *Information and Software Technology (IST)*. February 2020.

[J16] Luis Melo, Igor Wiese, and **M. d'Amorim**. Using Docker to Assist Q&A Forum Users. In *IEEE Transactions on Software Engineering (TSE)*. December 2019.

[J17] Sabrina Souto and **M. d'Amorim**. Time-Space Efficient Regression Testing for Configurable Systems. In *Journal of Systems and Software (JSS)*, Volume 137, pages 733-746, 2018.

[J18] Andrei Rimsa, **M. d'Amorim**, Fernando M. Q. Pereira, and Roberto S. Bigonha. Efficient Static Checker for Tainted Variable Attacks. *Science of Computer Programming*. Volume 80, pages 91–105, Feb. 2014

[J19] A. Sobeih, **M. d'Amorim**, M. Viswanathan, D. Marinov, and J. Hou. Assertion checking in J-Sim simulation models of network protocols. In *Transactions of The Society for Modeling and Simulation International (Simulation)*. Volume 86, Number 11, 651-673, November 2010.

[J20] M. Takaki, D. Cavalcanti, R. Gheyi, J. Iyoda, **M. d'Amorim**, R. Prudencio. Randomized Constraint Solvers: A comparative study. In *Innovations in Systems and Software Engineering: a NASA journal (ISSE)*. Volume 6, Number 3, 243-253, September 2010.

[J21] **M. d'Amorim**, S. Lauterburg and D. Marinov. Delta Execution for Efficient State-Space Exploration. In *IEEE Transactions on Software Engineering (TSE)*, Vol. 34, No. 5, pages 597-613, October 2008.

[J22] **M. d'Amorim** and G. Roşu. An Equational Specification for the Scheme Language. In *Journal of Universal Computer Science (JUCS)*, 11(7), pages 1327-1348, July 2005.

Conference Papers

[C1] M. M. Abid Naziri, Shinhae Kim, Feiran (Alex) Qin, **M. d'Amorim**, and Saikat Dutta. Testing Deep Learning Libraries via Neurosymbolic Constraint Learning (Centaur). In *International Conference on Software Engineering (ICSE)*. 2026.

[C2] Feiran (Alex) Qin, M. M. Abid Naziri, Hengyu Ai, Saikat Dutta, and **M. d'Amorim**. How Effective Is Coverage-Guided Fuzzing to Test Deep Learning Library APIs? (FlashFuzz). In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 2026.

[C3] Lingjun Liu, Feiran (Alex) Qin, Owolabi Legunsen, and **M. d'Amorim**. Learning Compiler Fuzzing Mutators from Historical Bugs. In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2026.

[C4] Denini Silva, MohamadAli Farahat, and **M. d'Amorim**. An Empirical Analysis of Portability Issues in Python Projects. In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 2026.

[C5] Pengyue Jiang, Kevin Guan, Mahdi Khosravi, Moustafa Ismail, **M. d'Amorim**, and Owolabi Legunsen. Fine-Grained Analyses for Evolution-Aware Runtime Verification. In *International Conference on Software Engineering (ICSE)*. 2026.

[C6] Facundo Molina, M. M. Abid Naziri, Feiran (Alex) Qin, Alessandra Gorla, and **M. d'Amorim**. Improving Deep Learning Library Testing with Machine Learning. In *IEEE/ACM International Conference on Automation of Software Test (AST)*. 2026.

[C7] M. M. Abid Naziri, Stefano Carlo Lambertenghi, Andrea Stocco, and **M. d'Amorim**. Misbehavior Forecasting for Focused Autonomous Driving Systems Testing. In *International Conference on Software Engineering (ICSE)*. 2026.

[C8] Kevin Guan, **M. d'Amorim**, and Owolabi Legunsen. Faster Explicit-Trace Monitoring-Oriented Programming for Runtime Verification. In *ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 2025.

[C9] M. M. Abid Naziri, Aman Kumar Singh, Benjamin Wu, Feiran (Alex) Qin, Saikat Dutta, and **M. d'Amorim**. BugsInDLLs: A Database of Reproducible Bugs in Deep Learning Libraries. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2025.

[C10] Ishrak Hayet, Andrew Scott, and **M. d'Amorim**. Feedback-Directed Partial Execution. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 2024.

[C11] U. Kulsum, Haotian Zhu, Bowen Xu, and **M. d'Amorim**. A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback. In *ACM International Conference on AI-Powered Software (AIware)*. 2024.

[C12] Andrea Stocco, Paulo J. Nunes, **M. d'Amorim**, and Paolo Tonella. ThirdEye: Attention Maps for Safe Autonomous Driving Systems. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. October 2022.

[C13] Sofia Reis, Rui Abreu, **M. d'Amorim**, and Daniel Fortunato. Leveraging Practitioners' Feedback to Improve a Security Linter. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*. October 2022.

[C14] Facundo Molina, Nazareno Aguirre, **M. d'Amorim**. Fuzzing Class Specifications. In *International Conference on Software Engineering (ICSE)*. May 2022.

[C15] Shouvick Mondal, Denini Silva, and **M. d'Amorim**. Soundy Automated Parallelization of Test Execution. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*. September 2021.

[C16] Daniela Costa, Carlos Mello, and **M. d'Amorim**. A Comparative Study on Methods and Tools for Handwritten Mathematical Expression Recognition. In *ACM Symposium on Document Engineering (DocEng)*. August 2021.

[C17] Jordan Henkel, Denini Silva, Leopoldo Teixeira, **M. d'Amorim**, and Thomas Reps. Shipwright: A Human-in-the-Loop System for Dockerfile Repair. In *International Conference on Software Engineering (ICSE)*. May 2021.

[C18] Leopoldo Teixeira, Breno Miranda, Henrique Rebêlo, and **M. d'Amorim**. Demystifying the Challenges to Formally Specifying API Properties for Runtime Verification. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. April 2021.

[C19] Breno Miranda, Igor Lima, Owolabi Legunsen, and **M. d'Amorim**. Prioritizing Runtime Verification Violations. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. pages 297-308, October 2020.

[C20] Marcio A. Guimarães, Leo Fernandes, Márcio Ribeiro, **M. d'Amorim**, and Rohit Gheyi. Optimizing Mutation Testing by Discovering Dynamic Mutant Subsumption Relations. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. pages 198–208, October 2020.

[C21] Denini Silva, Leopoldo Teixeira and **M. d'Amorim**. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *International Conference on Software Maintenance and Evolution (ICSME)*. pages 492–502, September 2020.

[C22] Gustavo Pinto, Breno Miranda, Supun Dissanayake, **M. d'Amorim**, Christoph Treude, and Antonia Bertolino. What is the Vocabulary of Flaky Tests? In *International Conference on Mining Software Repositories (MSR)*. pages 492–502, July 2020.

[C23] **M. d'Amorim**, Rui M. Abreu, and Carlos Mello. Visual Sketching: From Image Sketches to Code. In *International Conference on Software Engineering New Ideas and Emerging Results (ICSE NIER)*. pages 101–104, July 2020.

[C24] Sofia Reis, Rui Abreu, and **M. d'Amorim**. A Study of Demystifying the Combination of Dynamic Slicing and Spectrum-based Fault Localization. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4760-4766, Aug 2019.

[C25] Xiangyu Li, **M. d'Amorim**, and Alessandro Orso. Intent-Preserving Test Repair. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 141-152, April 2019.

[C26] Mattia Fazzini, Martin Prammer, **M. d'Amorim**, and Alessandro Orso. Automatically Translating Bug Reports into Test Cases for Mobile Apps. In *International Symposium on Software Testing and Analysis (ISSTA)*, pages 141-152, July 2018.

[C27] Xiangyu Li, Shaowei Zhu, **M. d'Amorim**, and Alessandro Orso. Enlightened Debugging. In *International Conference on Software Engineering (ICSE)*, pages 82-92, May 2018.

[C28] Jeanderson Candido, Luis Melo, **M. d'Amorim**. Test Suite Parallelization in Open-Source Projects: a Study on its Usage and Impact. In *IEEE/ACM Intl. Conference on Automated Software Engineering (ASE)*, pages 838-848, Nov. 2017.

[C29] Sabrina Souto, **M. d'Amorim**, Rohit Gheyi. Balancing Soundness and Efficiency for Practical Testing of Configurable Systems. In *International Conference on Software Engineering (ICSE)*, pages 632-642, May 2017.

[C30] Alexandre Perez, Rui Abreu, **M. d'Amorim**. Prevalence of Single-Fault Fixes and its Impact on Fault Localization. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 12-22, March 2017.

[C31] Xiangyu Li, **M. d'Amorim**, Alessandro Orso. Iterative User-Driven Fault Localization. In *Haifa Verification Conference (HVC)*, pages 82-98, November 2016.

[C32] Paulo Barros, René Just, Suzanne Millstein, Paul Vines, Werner Dietl, **M. d'Amorim**, and Michael D. Ernst. Static Analysis of Implicit Control Flow: Resolving Java Reflection and Android Intents. In *IEEE/ACM Intl. Conference on Automated Software Engineering (ASE)*, pages 669-679, November 2015.

[C33] Mateus Borges, Antonio Filieri, **M. d'Amorim**, and Corina S. Păsăreanu. Iterative Distribution-Aware Sampling for Probabilistic Software Analysis. In *European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 866-877, Aug-Sept 2015.

[C34] Sabrina Souto, Divya Gopinath, **M. d'Amorim**, Darko Marinov, Sarfraz Khurshid and Don Batory. Faster Bug Detection for Software Product Lines with Incomplete Feature Models. In *International Systems and Software Product Line Conference (SPLC)*, pages 151–160, July 2015.

[C35] Tianhai Liu, Mateus Borges, **M. d'Amorim**, and Mana Taghidiri. A Comparative Study of Incremental Constraint Solving Approaches in Symbolic Execution. *Haifa Verification Conference (HVC)*, pages 284-299, November 2014.

[C36] Quoc-Sang Phan, Pasquale Malacaria, Corina S. Păsăreanu, and **M. d'Amorim**. Quantifying Information Leaks using Reliability Analysis. *International SPIN Symposium on Software Model Checking*, pages 105-108, July 2014.

[C37] Mateus Borges, Antonio Filieri, **M. d'Amorim**, Corina S. Păsăreanu, and Willem Visser. Compositional Solution Space Quantification for Probabilistic Software Analysis. *ACM/SIGPLAN Programming Language Design and Implementation (PLDI)*, pages 123-132, June 2014.

[C38] José Carlos de Campos, Rui Abreu, Gordon Fraser, and **M. d'Amorim**. Entropy-based Test Generation for Improved Fault Localization. In *IEEE/ACM International Conference on Automated Software Engineering*, pages 257–267, November 2013.

[C39] Chang Hwan Peter Kim, Darko Marinov, Sarfraz Khurshid, Don Batory, Sabrina Souto, Paulo Barros, and **M. d'Amorim**. SPLat: Lightweight Dynamic Analysis for Reducing Combinatorics in Testing Configurable Systems. In *European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 257–267, August 2013.

[C40] Mateus Borges, **M. d'Amorim**, Saswat Anand, David Bushnell, and Corina S. Păsăreanu. Symbolic Execution with Interval Solving and Meta-heuristic Search. In *IEEE International Conference on Software Testing, Verification, and Validation (ICST)*, pages 111-120, April 2012.

[C41] Elton Alves, Milos Gligoric, Vilas Jagannath, and **M. d'Amorim**. Improved Lightweight Debugging with Dynamic Slicing and Change Data. In *IEEE/ACM International Symposium on Automated Software Engineering (ASE)*, pages 520-523, November 2011.

[C42] Matheus Souza, Mateus Borges, **M. d'Amorim**, and Corina S. Păsăreanu. CORAL: Solving Complex Constraints in Symbolic PathFinder. In *Proc. of the NASA Formal Methods Symposium (NFM)*, pages 359-374, April 2011.

[C43] Andrei Rimsa, **M. d'Amorim**, Fernando M. Q. Pereira. Efficient Tainted Flow Analysis. In *ETAPS Intl. Conference on Compiler Construction (CC)*, pages 124-143, March 2011.

[C44] Andrei Rimsa, **M. d'Amorim**, Fernando M. Q. Pereira. Efficient Static Checker for Tainted Variable Attacks. In *Brazilian Symposium on Programming Languages (SBLP)*, September 2010.

[C45] M. Takaki, D. Cavalcanti, R. Gheyi, J. Iyoda, **M. d'Amorim**, R. Prudencio. A Comparative Study of Randomized Constraint Solvers for Random-Symbolic Testing. In *NASA Formal Methods Symposium (NFM)*, pages 56-65, April 2009.

[C46] C. Bertolini, G. Peres, **M. d'Amorim**, A. Mota. An Empirical Evaluation of Automated Black Box Testing Techniques for Crashing GUIs. In *IEEE International Conference on Software Testing, Verification, and Validation (ICST)*, pages 21-30, April 2009.

[C47] **M. d'Amorim**, S. Lauterburg, and D. Marinov. Delta Execution for Efficient State-Space Exploration of Object-Oriented Programs. In *ACM/SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 50-60. July 2007.

[C48] **M. d'Amorim**, C. Pacheco, T. Xie, D. Marinov, and M. D. Ernst. An empirical comparison of automated generation and classification techniques for object-oriented unit testing. In *IEEE/ACM International Symposium on Automated Software Engineering (ASE)*, pages 59-68, September 2006.

[C49] **M. d'Amorim**, A. Sobeih, and D. Marinov. Optimized execution of deterministic blocks in Java PathFinder. In *International Conference on Formal Engineering Methods (ICFEM)*, pages 549-567, November 2006.

[C50] **M. d'Amorim** and G. Roşu. Efficient Monitoring of ω-Languages. In *Intl. Conf. on Computer Aided Verification (CAV)*. pages 364-378, July 2005.

[C51] **M. d'Amorim** and G. Roşu. An Equational Specification for the Scheme Language. In *Simpósio Brasileiro de Linguagens de Programação (SBLP)*. pages 229-242, June 2005.

[C52] F. Chen, **M. d'Amorim**, and G. Roşu. A Formal Monitoring-based Framework for Software Development and Analysis. In *Proc. of the International Conference on Formal Engineering Methods (ICFEM)*, pages 357-372, November 2004.

Workshops, Tool Demonstrations, and Other

[W1] Sarah Elder, Alex Klevan, Ranindya Paramitha, **M. d'Amorim**, and Laurie Williams. Aggregating Security Measures from the Dependency Tree. In *ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED)*. 2025.

[W2] Sivana Hamer, **M. d'Amorim**, and Laurie Williams. Just Another Copy and Paste? Comparing the Security Vulnerabilities of ChatGPT-Generated Code and StackOverflow Answers. In *IEEE Security and Privacy Workshops (SPW)*. 2024.

[W3] Marcello Cordeiro, Denini Silva, Leopoldo Teixeira, Breno Miranda, and **M. d'Amorim**. Shaker: a Tool for Detecting More Flaky Tests Faster. In *IEEE/ACM International Conference on Automated Software Engineering (ASE Tool Demonstrations)*. November 2021.

[W4] Davino Mauro Junior, Luis Melo, Harvey Lu, **M. d'Amorim**, and Atul Prakash. A Study of Vulnerability Analysis of Popular Smart Devices Through Their Companion Apps. In *IEEE Workshop on the Internet of Safe Things (SafeThings)*, May 2019.

[W5] T. Gvero, M. Gligoric, S.Lauterburg, **M. d'Amorim**, D. Marinov, S. Khurshid State Extensions for Java PathFinder. In *International Conference on Software Engineering (ICSE Demo)*. pages 863-866, May 2008.

[W6] Y. Zhou, D. Marinov, W. Sanders, C. Zilles, **M. d'Amorim**, S. Lauterburg, R. Lefever, J. Tucek Delta Execution for Software Reliability. In *Workshop on Hot Topics in System Dependability (HotDep)*. June 2007.

[W7] F. Chen, **M. d'Amorim** and G. Roşu. Checking and Correcting Behaviors of Java Programs at Runtime with Java-MOP. In *5th Workshop on Runtime Verification (RV)*. pages 3-20, July 2005.

[W8] **M. d'Amorim** and K. Havelund. Event-Based Runtime Verification of Java Programs. In *ACM/SIGSOFT International Workshop on Dynamic Analysis (WODA)*. pages 15-21, July 2005.

[W9] **M. d'Amorim**, C. Nogueira, G. Santos, A. Souza, and P. Borba. Integrating Code Generation and Refactoring. In *Proc. of the Workshop on Generative Programming* (ECOOP event), June 2002.

[W10] **M. d'Amorim** and C. Ferraz. Designing Jini Distributed Services: A Framework to support the development of reliable component networks. In *Proc. of the Workshop on Language Mechanisms for Programming Software Components* (OOPSLA event), October 2001.